Software Architecture Document (SAD)

BudgetMind: AI Cashflow Forecaster & Crisis Advisor

Version: 1.1
Status: Publish
Lead Architect: Isaac Furqan
Date: Apr 25, 2026

1. Introduction

BudgetMind is a forward-looking financial health dashboard specifically designed to mitigate the "Cash Flow Gap" for Small and Medium Enterprises (SMEs). Unlike historical accounting tools, BudgetMind utilizes predictive modeling and Generative AI to provide liquidity forecasts up to 90 days in advance. This document outlines the technical architecture, data orchestration, and component interaction required to deliver the "CFO in a Box" experience.

2. Architectural Representation

The system follows a modular, three-tier architecture leveraging Python-based technologies for rapid iteration and high performance.

- **Presentation Layer:** Streamlit-based interactive web interface.
- **Intelligence Layer:** Google Gemini 1.5 Pro for NLP-driven crisis advisory and simulations.
- **Data & Processing Layer:** Pandas/NumPy core for time-series aggregation and financial logic.

3. Data Flow Architecture

The core value of BudgetMind lies in the transformation of raw transaction logs into actionable AI insights.

3.1 CSV Ingestion & Pre-processing

The data pipeline begins with the `utils/` module, which handles the ingestion of structured financial data.

1. **Ingestion:** The Streamlit `file_uploader` captures CSV files containing historical transactions and upcoming scheduled payments.
2. **Validation:** Data is passed to `forecaster.py`, where Pandas performs schema validation, type casting (e.g., ensuring date formats), and handles missing values.

3.2 Pandas Processing Layer

This layer calculates the foundational metrics used for both visualization and AI analysis.

- **Running Balance Logic:** Pandas aggregates daily inflows and outflows to compute a "Running Balance".
- **Threshold Comparison:** The logic compares the projected balance against a "Safe Operating Threshold" defined by the user.
- **Anomaly Identification:** The system flags dates where the balance drops below zero or the threshold, creating a "Crisis Context" payload for the AI engine.

3.3 Gemini 1.5 Pro API Integration

The `gemini_client.py` facilitates the interaction between structured financial data and natural language advice.

1. **Context Construction:** The system extracts a summary of the 90-day forecast, including specific shortfall dates and major upcoming expenses.

2. **Prompt Engineering:** This data is injected into a system prompt stored in the `prompts/` directory. The prompt instructs Gemini to act as a "CFO in a Box".
3. **Inference:** Gemini 1.5 Pro analyzes the patterns to identify causes (e.g., late invoices) and suggest mitigation strategies.
4. **Response Handling:** The AI returns natural language advice and, in "Decision Simulator" mode, updated parameters for the forecast.

4. Streamlit Frontend State Management

Streamlit's `st.session_state` is utilized to ensure a seamless, reactive user experience without losing calculated data during reruns.

State Key	Description	Lifecycle
`raw_data`	Stores the initial Pandas DataFrame after CSV upload.	Persists until a new file is uploaded.
`forecast_df`	Stores the processed 90-day trajectory, including "What-If" modifications.	Updated on every simulation trigger.
`ai_advisor_response`	Cached natural language insights from Gemini 1.5 Pro.	Persists until data or parameters change.
`simulation_params`	Dictionary of user-defined scenarios (e.g., hiring costs, date offsets).	Updated via NLP text-interface.

5. Component Design

5.1 Core Modules

- **`forecaster.py`:** Contains the mathematical engine for time-series aggregation and balance projections.
- **`gemini_client.py`:** Manages the Generative AI SDK, handling authentication and temperature settings for structured output.
- **`visualizer.py`:** Utilizes Altair/Plotly to render interactive charts that sync with the current `session_state`.

5.2 Simulation Logic (What-If Analysis)

The "Decision Simulator" utilizes Gemini 1.5 Pro to parse natural language queries (e.g., "What if I hire a dev for $4k?"). The AI extracts the cost and date, which are then passed back to the Pandas layer to re-calculate the `forecast_df` in real-time.

6. Technical Stack Summary

- **Frontend:** Streamlit (v1.x).
- **Data Science:** Pandas, NumPy.
- **AI Engine:** Google Gemini 1.5 Pro (Google Generative AI SDK).
- **Visualization:** Plotly / Altair.
- **Deployment:** Containerized (Docker) for cloud-agnostic scalability.

7. Security & Compliance

- **Data Privacy:** Transaction data is processed in-memory and not stored on the local server post-session.
- **API Security:** Gemini API keys are managed via environment variables and never exposed to the client-side code.